

EAI (22EC752) — Study Priority

Embedded AI | CIE-1 | Built directly from your confirmed CIE-1 paper (24/09/2025, 09:45–11:30 AM) + EAI_TB (Deep Learning on Microcontrollers) | Both sides of all 5 questions included, since either could be asked

Q1 — Intelligent IoT Systems (10M) · CO1

(a) Intelligent IoT System vs. Cloud-based IoT System

IoT devices need high-speed connectivity but also low power, untethered, battery-run operation — a fundamental conflict. Most IoT devices use low-bit-rate protocols (Bluetooth Low Energy, LoRa, NB-IoT). If bandwidth isn't a constraint, a cloud-based IoT system simply sends all data to the cloud for inference — this is how most IoT systems work today, but it's the main reason AI adoption on IoT devices has been slow.

An Intelligent IoT (IIoT) system runs the AI workload directly on the low-power device itself. Motivations beyond power: privacy (voice/image data never leaves the device), and latency/reliability (internet packets have no guaranteed arrival time — unsuitable for mission-critical, fast-reaction cases like autonomous vehicles).

Most IoT systems are built around a Microcontroller Unit (MCU) — a low clock rate, low on-chip RAM chip with limited compute at low power/price. This class of devices running AI is called TinyML.

Types of ML Systems / Representation of Data

- Supervised, Unsupervised, Reinforcement learning — the three broad ML categories, applicable to data collected from IoT sensors.
- Data is represented as feature vectors/tables from sensor readings (e.g. accelerometer, temperature), feeding the ML model on-device or in the cloud.

OR — (b) Key Applications for Intelligent IoT Systems

- Smart agriculture: reduces pesticide/fertilizer overuse via targeted application; agricultural robots for planting/harvesting; pest detection via insect sound; weed detection via image recognition (cuts weedicide use >90%).
- Smart appliances: gesture/voice-controlled TVs, fridges that detect food-spoilage odors.
- Smart cities: drones for illegal parking detection, facial recognition for public safety.
- Smart health: wearables predicting heart attacks/epilepsy episodes, sleep monitoring.
- Smart homes: thermostats, power meters, sprinklers that reduce carbon footprint by activating only when needed.
- Smart industry: predictive maintenance via vibration/acoustic signal changes, barcode-reading robots, warehouse navigation via image recognition.

Q2 — ML/DL Tools, Frameworks & Deployment (10M) · CO1

(a) Tools/Frameworks for Developing & Deploying ML/DL Models on Microcontrollers

A typical MC has ~128 KB RAM and 1 MB flash, vs. a mobile phone's 4–6 GB RAM — so ML pipelines built for edge/mobile devices are NOT directly deployable on MCs; a methodical workflow and specialized software tools are needed.

- Python, Jupyter Notebooks, Google Colab: development environments with ML libraries (TensorFlow, Keras, PyTorch) for fast model development.
- TensorFlow (TF): Google's open-source framework, ~400MB — too large for MCs.
- TensorFlow Lite (TFLite): scaled-down TF for mobile/embedded/IoT, ~1MB binary, on-device inference. Benefits: low latency (no server round-trip), data privacy, no connectivity needed, small model size, low power.
- TensorFlow Lite Micro: further compressed for MCs with only KBs of memory (core runtime ~16KB on Arm Cortex M3); runs on bare metal, no OS/dynamic memory needed.

- AIMET (AI Model Efficiency Toolkit): quantizes/compresses PyTorch/TF models for lower power, storage, memory bandwidth, with minimal accuracy loss.
- Caffe: C++ DL framework (BAIR) for CNN/RCNN/LSTM, GPU/CPU accelerated, fast (60M images/day on one NVIDIA K40), but lacks a high-level API.
- CoreML: Apple's on-device ML framework, optimizes CPU/GPU/Neural Engine usage.
- ONNX: open interchange format letting models move between frameworks (e.g. PyTorch-trained → CoreML-deployed) without vendor lock-in.
- OpenVINO: Intel's toolkit to deploy/optimize inference engines from Caffe/PyTorch/TF models onto Intel hardware (cloud, on-prem, mobile).
- PyTorch / PyTorch Mobile: Facebook's framework with auto-differentiation (fast backprop); very flexible, popular with researchers.

Steps to run a TF model on a microcontroller: (1) Develop a compact TF model with supported ops (2) Convert to TFLite via the TFLite converter (3) Convert to a C byte-array and place in ROM (4) Use the C++ library to run inference on-device.

OR — (b) ML Model Deployment & Inferencing on Different Platforms

The main trend: move ML inference to edge devices, enabling quick access to real-time sensor data for fast response — giving devices human-like reactive intelligence. Running ML on MCs gives 3 advantages: latency, power consumption, privacy.

- Cloud intelligence: both training AND inference happen entirely in the cloud.
- Edge computing: mixed cloud-edge scenarios — cloud-based training, device-level inferencing (co-training/data offloading possible); frameworks like Federated Learning (FL) improve privacy/security in this setup.
- On-device/TinyML inference: both training and inference are constrained — in practice, only inference runs on MCs today; MC-based training (e.g. via FL) is still basic research.

Key fact to cite: "Both ML model training and inferencing is possible on edge devices, but only inferencing is possible on Microcontrollers."

Q3 — TinyML Hardware Boards & Compression (10M) · CO2

(a) Benefits of TinyML Hardware Boards + Major MC-based/TinyML Boards

TinyML hardware boards are compact, low-power, single-board computers that run ML algorithms at the edge, containing an MCU as CPU plus AI-acceleration technology, without needing a separate AI accelerator.

Benefits

- Small size & low power — suited to constrained deployments.
- Cost-effective — cheaper than traditional ML hardware.
- Easy to use — user-friendly, shortens developer learning curve.
- Flexible — usable across IoT, wearables, drones, robotics.
- Strong performance — many support deep learning/neural nets despite small size.

Major MC-based & TinyML Hardware Boards

- Arduino Nano 33 BLE Sense: Cortex-M4F processor + BLE, popular for TinyML.
- Arduino Nicla Sense ME: smallest form factor, industrial-grade sensors packed in.
- Raspberry Pi: versatile, low-cost, general-purpose.
- Adafruit Circuit Playground Express, EdgeBadge, SparkFun Edge, Coral Edge TPU, Syntiant TinyML board: additional common examples.

OR — (b) TinyML Model Compression Frameworks

Why needed: memory/storage limits on low-power devices; long inference time hurts real-time apps; costly to train/deploy large models; oversized models may not even fit some platforms.

- Quantization: reduces bits used for weights/activations (e.g. 32-bit float → 8-bit int). Weight quantization and Activation quantization are the two types; can be applied during or after training. Benefit: faster inference, less memory, lower power — trade-off with accuracy.
- Pruning: removes unneeded weights/parameters. Types: weight pruning (zero out individual weights), unit pruning (remove whole neurons), structure pruning (remove whole layers). Improves interpretability and generalization too.
- Low-rank approximation: decomposes large/sparse matrices into smaller ones (matrix/tensor decomposition) to reduce model size and computation.
- Knowledge distillation: a large “teacher” model's knowledge is transferred to a smaller “student” model, which is trained to mimic the teacher's predictions — similar performance, fewer parameters, faster inference.

TensorFlow Lite's compression techniques: Post-Training Quantization (Float16, Hybrid, Integer), During-Training Quantization, Post-Training Pruning, Post-Training Clustering.

Q4 — Jetson Nano / Microcontrollers with AI Accelerators (10M) · CO2

(a) NVIDIA Jetson Nano — Features & Support for TinyML

A compact, powerful computer for ML/AI, built around the Jetson Xavier processor. Well-suited for image/video processing, robotics, and IoT projects; popular for its low cost + high performance.

- 128-core NVIDIA Maxwell GPU
- Quad-core Arm Cortex-A57 CPU
- 4 GB LPDDR4 memory
- HDMI 2.0 and eDP 1.4 display outputs; MIPI CSI-2 camera connector

Software support: JetPack SDK with support for TensorFlow and PyTorch, plus libraries/tools for computer vision and robotics.

OR — (b) Microcontrollers and Microcontroller with AI Accelerators

A microcontroller (MC) is a small, low-power computer with processor+memory+I/O on one chip, embedded in appliances/automobiles/industrial equipment.

An AI accelerator is hardware built into an MC specifically to speed up ML/AI algorithm execution — used for facial recognition, speech recognition, intelligent sensing/control — while staying energy-efficient.

Advantages of Microcontrollers

- Power: milliwatt/microwatt-level — $\sim 1000\times$ more energy-efficient than a typical computer.
- Price: very low cost (>28 billion MCs shipped in 2020).
- Multifunctional: usable in virtually all devices/gadgets/appliances.

Classification of Microcontrollers

- By architecture: 8-bit (cheapest, simple/low-power), 16-bit (more performance, e.g. automobiles), 32-bit (advanced, e.g. medical/military systems).
- By memory size, clock speed, I/O capabilities — additional classification dimensions.

Example AI-Accelerator MC Hardware (name-drop for marks)

- Google Coral Edge TPU: dedicated TPU for edge ML, compatible with TF/TFLite.
- Qualcomm QCS605: Kryo CPU + Adreno GPU + built-in AI engine, on-device ML for image/voice recognition without cloud.
- NXP i.MX 8M: Arm Cortex-A53-based, low-cost edge computing/on-device AI.
- STMicroelectronics STM32L4: Arm Cortex-M4 core, up to 80MHz, 1MB flash/320KB SRAM, DSP instructions for audio/signal processing, advanced low-power modes.

Q5 — EML Systems & Data Engineering (10M) · CO3

(a) Basic Blocks Frequently Utilized in EML Systems + Examples

A typical Embedded Machine Learning (EML) system = a small processor/MC + sensors/actuators + a pre-trained ML algorithm tailored for low power and rapid execution.

Basic Blocks

- Sensors and Hardware: collect environmental data (e.g. accelerometer, temperature sensor); may need MCs, DSPs, FPGAs, or ASICs.
- Operating System: lightweight OS suited to constrained resources — FreeRTOS, VxWorks, μ C/OS.
- Communication Interfaces: Ethernet, USB, Bluetooth, Wi-Fi for interfacing with other devices/systems.
- ML Model Development & Deployment: data preprocessing → model training/evaluation → model deployment.

Key Features of EML Systems

- Hardware integration, specialized single-task design, resource-constrained by design, and can operate offline.

Examples of EML Systems

- Smart surveillance cameras — identify/categorize objects in view.
- Predictive maintenance for industrial equipment via sensor data analysis.
- Agricultural sensor networks/drones — soil moisture/temperature + pest/crop monitoring.
- Smart home devices — thermostats, security systems adapting to user habits.
- Autonomous vehicles — camera/LiDAR/radar-based navigation, brake prediction.
- Healthcare devices — smartwatches, insulin pumps adjusting treatment in real time.

Pros: low latency, increased efficiency, improved privacy, better resource utilization, enhanced UX. Cons: limited capabilities, difficult to update (needs physical access), hardware-dependent, limited flexibility.

OR — (b) Data Engineering Steps for TinyML

Data engineering = preparing data for analysis and modeling. Main aspects:

1. Cleaning

- Identify/correct errors (typos, wrong values).
- Handle missing values (default value, remove rows, or impute with mean/median).
- Normalize/standardize data (mean 0, std 1) to improve ML algorithm efficiency.
- Remove unnecessary columns.
- Check for and handle outliers carefully (they can significantly affect model output).
- Convert data types so the data is readable by the ML program.

2. Organizing

Structure the data so analysis/modeling can be done efficiently and effectively — well-organized data is easier to work with and improves model accuracy/performance.

3. Transformation

Convert data into a format/structure suitable for the ML model (e.g. raw text → CSV, image → numerical array). Common transformations:

- Normalization: adjust data to mean 0, std 1.
- Encoding: convert categorical data to numeric (one-hot encoding, label encoding).
- Feature extraction: pull useful info from raw data (e.g. image edges, audio spectral features) — reduces dimensionality.
- Imputation: estimate/replace missing or incomplete data.
- Data deduplication: remove repeated data to prevent overfitting.

Other processes under data engineering: feature engineering (creating new features) and data integration (combining data from multiple sources).